

State of the Art Node.js

Die Plattform verstehen, nicht nur benutzen

Sebastian Springer

Agenda

09:30	Einstieg: Node.js heute
09:45	**Teil 1 –** Die Plattform: ESM, TypeScript, Test Runner, Permissions, Diagnostik
10:45	**Teil 2 –** Kerntechnologien: Event Loop, Timers, Events, Streams
11:00	*Pause*
11:30	**Teil 2 –** Worker Threads, asynchrone Kontrollflüsse
12:00	**Teil 3 –** Architektur: Module, Fehler, Concurrency, Observability, Schichten
12:45	Zusammenfassung & Fragen

Mitmachen

Alle Beispiele laufen ohne Dependencies

```
node/
├── examples/    ← fertige Beispiele
└── live/        ← gleiche Dateien mit TODOs
```

- Node.js **≥ 24** (LTS)
- Jede Datei startet mit einem Kommentar, *wie* man sie ausführt
- `live/` enthält nummerierte Platzhalter: `// TODO 2: …`
- Die Schritte entsprechen den `#region`-Blöcken in `examples/`

```
node --version
```

Node.js heute

Von der Runtime zur Plattform

- Gerade Versionen werden **LTS** – produktiv nur LTS einsetzen
- Ungerade Versionen: Spielwiese für neue Features
- Web-Standards sind angekommen: `fetch`, Web Streams, `WebSocket`, `URLPattern`, `AbortController`
- Viele Aufgaben brauchen **keine npm-Pakete** mehr

> **Faustregel**
> Jede Dependency ist Code, den ihr wartet, prüft und absichert. Was die Plattform kann, sollte die Plattform machen.

Weniger Dependencies

Früher	Heute in Node.js
dotenv	`--env-file`, `process.loadEnvFile()`
nodemon	`node --watch`
ts-node, tsx (für einfache Fälle)	Type Stripping: `node app.ts`
jest, mocha, sinon	`node:test`, `node:assert`, `mock`
node-fetch, axios	`fetch` (undici)
yargs, commander (einfache CLIs)	`util.parseArgs`
chalk	`util.styleText`
glob, uuid	`fs.glob`, `crypto.randomUUID`
better-sqlite3	`node:sqlite`

Teil 1

Die Plattform

ESM · TypeScript · Tooling · Test Runner · Permissions · Diagnostik

ES Modules sind der Standard

```
// package.json
{
  "type": "module",
  "engines": { "node": ">=24" }
}
```

▸ examples/01-platform/01-esm

- Dateiendungen sind Pflicht: `./math.js`
- Built-ins immer mit `node:`-Präfix
- `.cjs` / `.mjs` erzwingen das Format pro Datei
- Top-level `await` funktioniert
- Kein `__dirname`, `require`, `module`

import.meta `20.11+`

```
1 console.log(import.meta.filename);  // /…/01-esm/index.js
2 console.log(import.meta.dirname);   // /…/01-esm
3 console.log(import.meta.url);       // file:///…/01-esm/index.js
4 console.log(import.meta.main);      // true, wenn Einstiegspunkt (24.2+)
5 console.log(import.meta.resolve('./math.js'));
```

> **Kein Workaround mehr**
>
> `fileURLToPath(new URL('.', import.meta.url))` ist Geschichte.

▸ examples/01-platform/01-esm/index.js · Schritt 1

import.meta `20.11+`

```
1 console.log(import.meta.filename);  // /…/01-esm/index.js
2 console.log(import.meta.dirname);   // /…/01-esm
3 console.log(import.meta.url);       // file:///…/01-esm/index.js
4 console.log(import.meta.main);      // true, wenn Einstiegspunkt (24.2+)
5 console.log(import.meta.resolve('./math.js'));
```

> **Kein Workaround mehr**
>
> `fileURLToPath(new URL('.', import.meta.url))` ist Geschichte.

▸ examples/01-platform/01-esm/index.js · Schritt 1

import.meta `20.11+`

```
1 console.log(import.meta.filename);  // /…/01-esm/index.js
2 console.log(import.meta.dirname);   // /…/01-esm
3 console.log(import.meta.url);       // file:///…/01-esm/index.js
4 console.log(import.meta.main);      // true, wenn Einstiegspunkt (24.2+)
5 console.log(import.meta.resolve('./math.js'));
```

> **Kein Workaround mehr**
>
> `fileURLToPath(new URL('.', import.meta.url))` ist Geschichte.

▸ examples/01-platform/01-esm/index.js · Schritt 1

import.meta `20.11+`

```
1 console.log(import.meta.filename);  // /…/01-esm/index.js
2 console.log(import.meta.dirname);   // /…/01-esm
3 console.log(import.meta.url);       // file:///…/01-esm/index.js
4 console.log(import.meta.main);      // true, wenn Einstiegspunkt (24.2+)
5 console.log(import.meta.resolve('./math.js'));
```

> **Kein Workaround mehr**
>
> `fileURLToPath(new URL('.', import.meta.url))` ist Geschichte.

▸ `examples/01-platform/01-esm/index.js · Schritt 1`

JSON-Import & dynamisches Laden

```
1  // Import Attributes – Typ wird explizit angegeben
2  import config from './config.json' with { type: 'json' };
3
4  // Top-level await + dynamischer Import
5  const { add } = await import('./math.js');
6
7  // Optionale Abhängigkeiten nur bei Bedarf laden
8  const { styleText } = config.colors
9    ? await import('node:util')
10   : { styleText: (_, text) => text };
```

▸ examples/01-platform/01-esm/index.js · Schritt 2–3

JSON-Import & dynamisches Laden

```
1  // Import Attributes – Typ wird explizit angegeben
2  import config from './config.json' with { type: 'json' };
3
4  // Top-level await + dynamischer Import
5  const { add } = await import('./math.js');
6
7  // Optionale Abhängigkeiten nur bei Bedarf laden
8  const { styleText } = config.colors
9    ? await import('node:util')
10   : { styleText: (_, text) => text };
```

▸ examples/01-platform/01-esm/index.js · Schritt 2–3

JSON-Import & dynamisches Laden

```
// Import Attributes – Typ wird explizit angegeben
import config from './config.json' with { type: 'json' };

// Top-level await + dynamischer Import
const { add } = await import('./math.js');

// Optionale Abhängigkeiten nur bei Bedarf laden
const { styleText } = config.colors
  ? await import('node:util')
  : { styleText: (_, text) => text };
```

▸ examples/01-platform/01-esm/index.js · Schritt 2–3

require(esm) `20.19+ / 22.12+`

```
// legacy.cjs – CommonJS lädt ein ES-Modul synchron
const math = require('./math.js');
math.multiply(6, 7);  // 42
math.default;         // { name: 'math', version: 1 }

// Grenze: Module mit top-level await
try {
  require('./index.js');
} catch (err) {
  err.code;  // 'ERR_REQUIRE_ASYNC_MODULE'
}
```

- Libraries können ESM-only ausliefern – CJS-Nutzer bleiben kompatibel
- Der „Dual Package Hazard“ verliert an Bedeutung

▸ examples/01-platform/01-esm/legacy.cjs · Schritt 5

require(esm) `20.19+ / 22.12+`

```cjs
// legacy.cjs – CommonJS lädt ein ES-Modul synchron
const math = require('./math.js');
math.multiply(6, 7);  // 42
math.default;         // { name: 'math', version: 1 }

// Grenze: Module mit top-level await
try {
  require('./index.js');
} catch (err) {
  err.code;  // 'ERR_REQUIRE_ASYNC_MODULE'
}
```

- Libraries können ESM-only ausliefern – CJS-Nutzer bleiben kompatibel
- Der „Dual Package Hazard“ verliert an Bedeutung

▸ examples/01-platform/01-esm/legacy.cjs · Schritt 5

TypeScript direkt ausführen `22.18+ / 23.6+`

```ts
import type { Greeting } from './types.ts';

interface User {
  id: number;
  name: string;
  roles: Role[];
}

function greet(user: User): Greeting {
  return `Hallo ${user.name}`;
}
```

```
$ node user.ts
Hallo Ada (admin, active)
```

▸ examples/01-platform/02-typescript/user.ts

TypeScript direkt ausführen `22.18+ / 23.6+`

```ts
import type { Greeting } from './types.ts';

interface User {
  id: number;
  name: string;
  roles: Role[];
}

function greet(user: User): Greeting {
  return `Hallo ${user.name}`;
}
```

```
$ node user.ts
Hallo Ada (admin, active)
```

▸ examples/01-platform/02-typescript/user.ts

TypeScript direkt ausführen `22.18+ / 23.6+`

```ts
import type { Greeting } from './types.ts';

interface User {
  id: number;
  name: string;
  roles: Role[];
}

function greet(user: User): Greeting {
  return `Hallo ${user.name}`;
}
```

```
$ node user.ts
Hallo Ada (admin, active)
```

▸ examples/01-platform/02-typescript/user.ts

Type Stripping – was passiert genau?

- Typen werden durch **Leerzeichen ersetzt** – Zeilen und Spalten bleiben gleich, keine Source Maps nötig
- **Kein Typecheck!** Dafür weiterhin `tsc --noEmit` in CI und Editor
- Nur „erasable syntax“: kein `enum`, `namespace`, Parameter Properties
- Imports mit `.ts`-Endung, Typen über `import type`

```
{
  "compilerOptions": {
    "module": "nodenext",
    "noEmit": true,
    "allowImportingTsExtensions": true,
    "erasableSyntaxOnly": true,
    "verbatimModuleSyntax": true
  }
}
```

```
// statt enum:
const Status = { Active: 'active' } as const;
type Status = (typeof Status)[keyof typeof Status];
```

▸ examples/01-platform/02-typescript/tsconfig.json

Developer Experience eingebaut

`node --env-file=.env app.js`	.env laden (`--env-file-if-exists` für optionale Dateien)
`process.loadEnvFile()`	dasselbe zur Laufzeit
`node --watch app.js`	Neustart bei Dateiänderungen
`node --run start`	package.json-Skripte – deutlich schneller als `npm run`, ohne pre/post-Hooks
`util.parseArgs()`	CLI-Argumente parsen
`util.styleText()`	Farbige Ausgabe, respektiert `NO_COLOR`

▸ examples/01-platform/03-cli

Eine CLI ohne Dependencies

```js
import { parseArgs, styleText } from 'node:util';

if (!process.env.GREETING) {
  process.loadEnvFile(join(import.meta.dirname, '.env'));
}

const { values, positionals } = parseArgs({
  options: {
    name: { type: 'string', short: 'n', default: 'Welt' },
    verbose: { type: 'boolean', short: 'v', default: false },
  },
  allowPositionals: true,
});

console.log(styleText(['bold', 'green'], `${process.env.GREETING}, ${values.name}!`));
```

```
$ node --run start
Servus, JSDays!
```

▸ examples/01-platform/03-cli/cli.js · Schritt 1–3

Eine CLI ohne Dependencies

```
import { parseArgs, styleText } from 'node:util';

if (!process.env.GREETING) {
  process.loadEnvFile(join(import.meta.dirname, '.env'));
}

const { values, positionals } = parseArgs({
  options: {
    name: { type: 'string', short: 'n', default: 'Welt' },
    verbose: { type: 'boolean', short: 'v', default: false },
  },
  allowPositionals: true,
});

console.log(styleText(['bold', 'green'], `${process.env.GREETING}, ${values.name}!`));
```

```
$ node --run start
Servus, JSDays!
```

▸ examples/01-platform/03-cli/cli.js · Schritt 1–3

Eine CLI ohne Dependencies

```js
import { parseArgs, styleText } from 'node:util';

if (!process.env.GREETING) {
  process.loadEnvFile(join(import.meta.dirname, '.env'));
}

const { values, positionals } = parseArgs({
  options: {
    name: { type: 'string', short: 'n', default: 'Welt' },
    verbose: { type: 'boolean', short: 'v', default: false },
  },
  allowPositionals: true,
});

console.log(styleText(['bold', 'green'], `${process.env.GREETING}, ${values.name}!`));
```

```
$ node --run start
Servus, JSDays!
```

▸ examples/01-platform/03-cli/cli.js · Schritt 1–3

Eine CLI ohne Dependencies

```
import { parseArgs, styleText } from 'node:util';

if (!process.env.GREETING) {
  process.loadEnvFile(join(import.meta.dirname, '.env'));
}

const { values, positionals } = parseArgs({
  options: {
    name: { type: 'string', short: 'n', default: 'Welt' },
    verbose: { type: 'boolean', short: 'v', default: false },
  },
  allowPositionals: true,
});

console.log(styleText(['bold', 'green'], `${process.env.GREETING}, ${values.name}!`));
```

```
$ node --run start
Servus, JSDays!
```

▸ examples/01-platform/03-cli/cli.js · Schritt 1–3

Der eingebaute Test Runner

- `node:test` + `node:assert` – stabil, schnell, keine Konfiguration
- Findet `*.test.js`, `test/**/*.js` automatisch
- Jede Testdatei läuft in einem eigenen Prozess
- Mocks, Mock Timers, Snapshots, Coverage, Reporter

▸ `examples/01-platform/04-test-runner`

```
node --test
node --test --watch
node --test --experimental-test-coverage
node --test --test-update-snapshots
node --test --test-name-pattern="TTL"
node --test --test-reporter=junit \
  --test-reporter-destination=report.xml
```

describe, it, assert

```js
import { describe, it, mock } from 'node:test';
import assert from 'node:assert/strict';
import { Cart } from './cart.js';

describe('Cart', () => {
  it('berechnet die Summe', () => {
    const cart = new Cart();
    cart.add('book', 20, 2);
    cart.add('pen', 2.5);

    assert.equal(cart.total, 42.5);
  });

  it('lehnt ungültige Mengen ab', () => {
    assert.throws(() => new Cart().add('book', 20, 0), RangeError);
  });
});
```

▸ examples/01-platform/04-test-runner/cart.test.js · Schritt 1

describe, it, assert

```
import { describe, it, mock } from 'node:test';
import assert from 'node:assert/strict';
import { Cart } from './cart.js';

describe('Cart', () => {
  it('berechnet die Summe', () => {
    const cart = new Cart();
    cart.add('book', 20, 2);
    cart.add('pen', 2.5);

    assert.equal(cart.total, 42.5);
  });

  it('lehnt ungültige Mengen ab', () => {
    assert.throws(() => new Cart().add('book', 20, 0), RangeError);
  });
});
```

▸ examples/01-platform/04-test-runner/cart.test.js · Schritt 1

describe, it, assert

```
import { describe, it, mock } from 'node:test';
import assert from 'node:assert/strict';
import { Cart } from './cart.js';

describe('Cart', () => {
  it('berechnet die Summe', () => {
    const cart = new Cart();
    cart.add('book', 20, 2);
    cart.add('pen', 2.5);

    assert.equal(cart.total, 42.5);
  });

  it('lehnt ungültige Mengen ab', () => {
    assert.throws(() => new Cart().add('book', 20, 0), RangeError);
  });
});
```

▸ examples/01-platform/04-test-runner/cart.test.js · Schritt 1

Mocks & Mock Timers

```
it('belastet den Payment-Service', async () => {
  const charge = mock.fn(async (amount) =>
    ({ id: 'tx-1', amount }));
  const cart = new Cart();
  cart.add('book', 20);

  await cart.checkout({ charge });

  assert.equal(charge.mock.callCount(), 1);
  assert.deepEqual(
    charge.mock.calls[0].arguments, [20]);
});
```

```
it('leert nach Ablauf der TTL', (t) => {
  t.mock.timers.enable({ apis: ['setTimeout'] });
  const onExpire = t.mock.fn();
  const cart = new Cart({ ttl: 1000, onExpire });
  cart.add('book', 20);

  t.mock.timers.tick(999);
  assert.equal(onExpire.mock.callCount(), 0);

  t.mock.timers.tick(1);
  assert.equal(onExpire.mock.callCount(), 1);
});
```

Mocks über `t.mock` werden nach dem Test automatisch zurückgesetzt.

▸ examples/01-platform/04-test-runner/cart.test.js · Schritt 2–3

Snapshots & Coverage

```
it('serialisiert den Warenkorb stabil', (t) => {
  const cart = new Cart();
  cart.add('book', 20, 2);
  t.assert.snapshot(cart.toJSON());   // → cart.test.js.snapshot
});
```

```
$ node --test --experimental-test-coverage
i file      | line % | branch % | funcs % | uncovered lines
i cart.js   | 100.00 |   100.00 |   90.00 |
```

▸ examples/01-platform/04-test-runner/cart.test.js · Schritt 4

Permission Model `stabil seit 22.13 / 23.5`

- Standard: Node darf **alles**, was der Benutzer darf
- Mit `--permission`: **nichts** außer dem Einstiegsskript
- Rechte werden gezielt freigegeben
- Ein „Sicherheitsgurt“ gegen Fehler und kompromittierte Dependencies – **keine** Sandbox für fremden Code

`--allow-fs-read=./data`

`--allow-fs-write=./tmp`

`--allow-child-process`

`--allow-worker`

`--allow-net` 25+

`--allow-addons`, `--allow-wasi`

▸ `examples/01-platform/05-permissions`

Permissions zur Laufzeit

```
if (process.permission) {
  process.permission.has('fs.read', dataDir);   // true / false
  process.permission.has('child');
}

await readFile(join(dataDir, 'config.json'));   // ERR_ACCESS_DENIED?
```

```
$ node --permission --allow-fs-read=./data app.js
Permission Model aktiv
  fs.read  data/ : true
  fs.write data/ : false
✔ Datei lesen { secret: 'nur mit --allow-fs-read lesbar' }
✘ Datei schreiben: ERR_ACCESS_DENIED FileSystemWrite …/data/out.txt
✘ Kindprozess: ERR_ACCESS_DENIED ChildProcess node
```

▸ examples/01-platform/05-permissions/app.js · Schritt 1–2

Diagnostik – der Werkzeugkasten

`node --inspect`	Chrome DevTools: Debugger, Profiler, Memory
`node --cpu-prof`	CPU-Profil als `.cpuprofile` beim Beenden
`node --heap-prof` / `v8.writeHeapSnapshot()`	Speicher analysieren, Leaks finden
`process.report`	JSON-Report: Stack, Heap, libuv-Handles, Umgebung
`monitorEventLoopDelay()`	Wie lange warten Callbacks auf die Event Loop?
`performance.mark/measure`	Eigene Messpunkte
`--trace-warnings`, `--trace-sync-io`	Woher kommt die Warnung? Wo blockiert sync I/O?

▸ `examples/01-platform/06-diagnostics`

Event Loop Delay messen

```
const loopDelay = monitorEventLoopDelay({ resolution: 10 });
loopDelay.enable();

setInterval(() => {
  const ms = (ns) => (ns / 1e6).toFixed(1);
  console.log(`p99=${ms(loopDelay.percentile(99))}ms max=${ms(loopDelay.max)}ms`);
  loopDelay.reset();
}, 2000).unref();
```

```
$ node load.js slow 5              $ node load.js slow-async 5
5x /slow: 95, 135, 199, 262, 325   5x /slow-async: 106, 88, 74, 80, 140
1x /fast parallel dazu: 326 ms     1x /fast parallel dazu: 8 ms
```

Ein synchroner Aufruf bremst alle Requests

`pbkdf2Sync` blockiert die Event Loop – `/fast` wartet 326 ms statt 8 ms.

▸ examples/01-platform/06-diagnostics/server.js · Schritt 1

Event Loop Delay messen

```
const loopDelay = monitorEventLoopDelay({ resolution: 10 });
loopDelay.enable();

setInterval(() => {
  const ms = (ns) => (ns / 1e6).toFixed(1);
  console.log(`p99=${ms(loopDelay.percentile(99))}ms max=${ms(loopDelay.max)}ms`);
  loopDelay.reset();
}, 2000).unref();
```

```
$ node load.js slow 5              $ node load.js slow-async 5
5x /slow: 95, 135, 199, 262, 325   5x /slow-async: 106, 88, 74, 80, 140
1x /fast parallel dazu: 326 ms     1x /fast parallel dazu: 8 ms
```

> **Ein synchroner Aufruf bremst alle Requests**
>
> `pbkdf2Sync` blockiert die Event Loop – `/fast` wartet 326 ms statt 8 ms.

▸ examples/01-platform/06-diagnostics/server.js · Schritt 1

Messen, Reports, Heap Snapshots

```
new PerformanceObserver((list) => {
  for (const entry of list.getEntries())
    console.log(`${entry.name}: ${entry.duration.toFixed(1)}ms`);
}).observe({ entryTypes: ['measure'] });

performance.mark('hash-start');
pbkdf2Sync('secret', 'salt', 300_000, 64, 'sha512');
performance.measure('hash', 'hash-start');

const report = process.report.getReport();
process.getActiveResourcesInfo();   // ['TCPServerWrap', 'TCPSocketWrap']
writeHeapSnapshot();                // → Heap.….heapsnapshot für DevTools
```

▸ examples/01-platform/06-diagnostics/server.js · Schritt 2–3

Messen, Reports, Heap Snapshots

```
new PerformanceObserver((list) => {
  for (const entry of list.getEntries())
    console.log(`${entry.name}: ${entry.duration.toFixed(1)}ms`);
}).observe({ entryTypes: ['measure'] });

performance.mark('hash-start');
pbkdf2Sync('secret', 'salt', 300_000, 64, 'sha512');
performance.measure('hash', 'hash-start');

const report = process.report.getReport();
process.getActiveResourcesInfo();   // ['TCPServerWrap', 'TCPSocketWrap']
writeHeapSnapshot();                // → Heap.….heapsnapshot für DevTools
```

▸ examples/01-platform/06-diagnostics/server.js · Schritt 2–3

Messen, Reports, Heap Snapshots

```
new PerformanceObserver((list) => {
  for (const entry of list.getEntries())
    console.log(`${entry.name}: ${entry.duration.toFixed(1)}ms`);
}).observe({ entryTypes: ['measure'] });

performance.mark('hash-start');
pbkdf2Sync('secret', 'salt', 300_000, 64, 'sha512');
performance.measure('hash', 'hash-start');

const report = process.report.getReport();
process.getActiveResourcesInfo();   // ['TCPServerWrap', 'TCPSocketWrap']
writeHeapSnapshot();                // → Heap.….heapsnapshot für DevTools
```

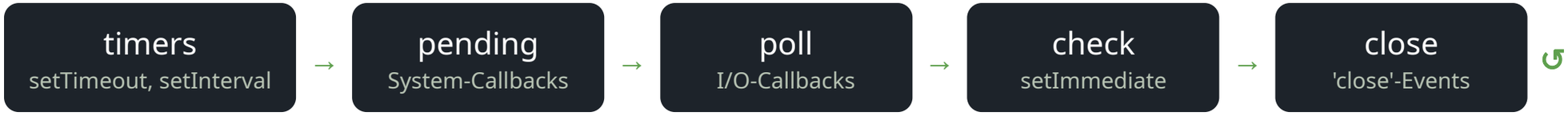
▸ examples/01-platform/06-diagnostics/server.js · Schritt 2–3

Teil 2

Kerntechnologien

Event Loop · Timers · Events · Streams · Worker Threads · Async-Kontrollfluss

Die Event Loop



nextTick-Queue (process.nextTick) | Microtask-Queue (Promises, queueMicrotask) … werden nach **jedem** Callback vollständig geleert

- JavaScript läuft in **einem** Thread – I/O übernehmen libuv und das Betriebssystem
- Jeder synchrone Callback blockiert alle anderen

Quiz: In welcher Reihenfolge?

```
console.log('sync 1');

setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));

process.nextTick(() => console.log('nextTick'));
Promise.resolve().then(() => console.log('promise'));

readFile(import.meta.filename, () => {
  console.log('I/O');
  setTimeout(() => console.log('timeout aus I/O'), 0);
  setImmediate(() => console.log('immediate aus I/O'));
});

console.log('sync 2');
```

▸ examples/02-core/01-event-loop/order.js · order.cjs

Quiz: In welcher Reihenfolge?

```js
console.log('sync 1');

setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));

process.nextTick(() => console.log('nextTick'));
Promise.resolve().then(() => console.log('promise'));

readFile(import.meta.filename, () => {
  console.log('I/O');
  setTimeout(() => console.log('timeout aus I/O'), 0);
  setImmediate(() => console.log('immediate aus I/O'));
});

console.log('sync 2');
```

```
sync 1
sync 2
promise          ← ESM!
nextTick
timeout          ← nicht garantiert
immediate        ← nicht garantiert
I/O
immediate aus I/O
timeout aus I/O
```

▸ examples/02-core/01-event-loop/order.js · order.cjs

Quiz: In welcher Reihenfolge?

```js
console.log('sync 1');

setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));

process.nextTick(() => console.log('nextTick'));
Promise.resolve().then(() => console.log('promise'));

readFile(import.meta.filename, () => {
  console.log('I/O');
  setTimeout(() => console.log('timeout aus I/O'), 0);
  setImmediate(() => console.log('immediate aus I/O'));
});

console.log('sync 2');
```

```
sync 1
sync 2
promise          ← ESM!
nextTick
timeout          ← nicht garantiert
immediate        ← nicht garantiert
I/O
immediate aus I/O
timeout aus I/O
```

ESM vs. CommonJS

Ein ES-Modul wird selbst als async Job ausgewertet – die Microtask-Queue läuft vor der nextTick-Queue. In `order.cjs` ist es umgekehrt.

▸ examples/02-core/01-event-loop/order.js · order.cjs

Timers als Promises

```mjs
import { setTimeout as sleep, setInterval, scheduler } from 'node:timers/promises';

const value = await sleep(300, 'fertig');

for await (const startedAt of setInterval(100, Date.now())) {
  console.log(`tick nach ${Date.now() - startedAt}ms`);
  if (++tick === 3) break;  // räumt den Timer automatisch auf
}

try {
  await sleep(5000, null, { signal: AbortSignal.timeout(200) });
} catch (err) {
  err.name;  // 'AbortError', err.cause.name: 'TimeoutError'
}
```

▸ examples/02-core/01-event-loop/timers.js · Schritt 1–3

Timers als Promises

```js
import { setTimeout as sleep, setInterval, scheduler } from 'node:timers/promises';

const value = await sleep(300, 'fertig');

for await (const startedAt of setInterval(100, Date.now())) {
  console.log(`tick nach ${Date.now() - startedAt}ms`);
  if (++tick === 3) break;  // räumt den Timer automatisch auf
}

try {
  await sleep(5000, null, { signal: AbortSignal.timeout(200) });
} catch (err) {
  err.name;  // 'AbortError', err.cause.name: 'TimeoutError'
}
```

▸ examples/02-core/01-event-loop/timers.js · Schritt 1–3

Timers als Promises

```js
import { setTimeout as sleep, setInterval, scheduler } from 'node:timers/promises';

const value = await sleep(300, 'fertig');

for await (const startedAt of setInterval(100, Date.now())) {
  console.log(`tick nach ${Date.now() - startedAt}ms`);
  if (++tick === 3) break;  // räumt den Timer automatisch auf
}

try {
  await sleep(5000, null, { signal: AbortSignal.timeout(200) });
} catch (err) {
  err.name;  // 'AbortError', err.cause.name: 'TimeoutError'
}
```

▸ examples/02-core/01-event-loop/timers.js · Schritt 1–3

Timers als Promises

```js
import { setTimeout as sleep, setInterval, scheduler } from 'node:timers/promises';

const value = await sleep(300, 'fertig');

for await (const startedAt of setInterval(100, Date.now())) {
  console.log(`tick nach ${Date.now() - startedAt}ms`);
  if (++tick === 3) break;  // räumt den Timer automatisch auf
}

try {
  await sleep(5000, null, { signal: AbortSignal.timeout(200) });
} catch (err) {
  err.name;  // 'AbortError', err.cause.name: 'TimeoutError'
}
```

▸ examples/02-core/01-event-loop/timers.js · Schritt 1–3

Die Event Loop atmen lassen

```js
for (let chunk = 0; chunk < 5; chunk++) {
  heavyWork(chunk);
  await scheduler.yield();   // andere Callbacks kommen dran
  await scheduler.wait(25);  // = sleep(25)
}

// Timer, die den Prozess nicht am Leben halten
const heartbeat = setInterval(sendHeartbeat, 1000);
heartbeat.unref();

// Variante in timers/promises
sleep(10_000, null, { ref: false });
```

Wann was?

Kurze Rechenarbeit aufteilen → `scheduler.yield()`. Echte CPU-Last → Worker Thread.

▸ examples/02-core/01-event-loop/timers.js · Schritt 4–5

Die Event Loop atmen lassen

```js
for (let chunk = 0; chunk < 5; chunk++) {
  heavyWork(chunk);
  await scheduler.yield();   // andere Callbacks kommen dran
  await scheduler.wait(25);  // = sleep(25)
}

// Timer, die den Prozess nicht am Leben halten
const heartbeat = setInterval(sendHeartbeat, 1000);
heartbeat.unref();

// Variante in timers/promises
sleep(10_000, null, { ref: false });
```

> **Wann was?**
> Kurze Rechenarbeit aufteilen → `scheduler.yield()`. Echte CPU-Last → Worker Thread.

▸ examples/02-core/01-event-loop/timers.js · Schritt 4–5

Die Event Loop atmen lassen

```
for (let chunk = 0; chunk < 5; chunk++) {
  heavyWork(chunk);
  await scheduler.yield();   // andere Callbacks kommen dran
  await scheduler.wait(25);  // = sleep(25)
}

// Timer, die den Prozess nicht am Leben halten
const heartbeat = setInterval(sendHeartbeat, 1000);
heartbeat.unref();

// Variante in timers/promises
sleep(10_000, null, { ref: false });
```

Wann was?

Kurze Rechenarbeit aufteilen → `scheduler.yield()`. Echte CPU-Last → Worker Thread.

▸ examples/02-core/01-event-loop/timers.js · Schritt 4–5

Events – synchron!

```
queue.on('order', (order) => console.log('  Listener:', order.id));

console.log('vor emit');
queue.emit('order', { id: 1 });
console.log('nach emit');
```

```
vor emit
  Listener: 1
nach emit
```

- `emit()` ruft alle Listener **sofort und nacheinander** auf
- Ein langsamer Listener bremst den Emitter
- Ein werfender Listener bricht `emit()` ab

▸ examples/02-core/02-events/events.js · Schritt 1

Fehler in Events

```
// 'error' ohne Listener → Exception → Prozess crasht
queue.on('error', (err) => console.log('Behandelt:', err.message));

// Beobachten, ohne zu behandeln (z. B. für Logging)
queue.on(errorMonitor, (err) => log(err));

// Async Listener: abgelehnte Promises landen sonst im Nirgendwo
class OrderQueue extends EventEmitter {
  constructor() { super({ captureRejections: true }); }
}
queue.on('order', async (order) => { throw new Error('ungültig'); });
queue.emit('order', { id: 99 });
// → landet im 'error'-Handler
```

▸ examples/02-core/02-events/events.js · Schritt 2 + 5

Fehler in Events

```
// 'error' ohne Listener → Exception → Prozess crasht
queue.on('error', (err) => console.log('Behandelt:', err.message));

// Beobachten, ohne zu behandeln (z. B. für Logging)
queue.on(errorMonitor, (err) => log(err));

// Async Listener: abgelehnte Promises landen sonst im Nirgendwo
class OrderQueue extends EventEmitter {
  constructor() { super({ captureRejections: true }); }
}
queue.on('order', async (order) => { throw new Error('ungültig'); });
queue.emit('order', { id: 99 });
// → landet im 'error'-Handler
```

▸ examples/02-core/02-events/events.js · Schritt 2 + 5

Fehler in Events

```
// 'error' ohne Listener → Exception → Prozess crasht
queue.on('error', (err) => console.log('Behandelt:', err.message));

// Beobachten, ohne zu behandeln (z. B. für Logging)
queue.on(errorMonitor, (err) => log(err));

// Async Listener: abgelehnte Promises landen sonst im Nirgendwo
class OrderQueue extends EventEmitter {
  constructor() { super({ captureRejections: true }); }
}
queue.on('order', async (order) => { throw new Error('ungültig'); });
queue.emit('order', { id: 99 });
// → landet im 'error'-Handler
```

▸ examples/02-core/02-events/events.js · Schritt 2 + 5

Fehler in Events

```
// 'error' ohne Listener → Exception → Prozess crasht
queue.on('error', (err) => console.log('Behandelt:', err.message));

// Beobachten, ohne zu behandeln (z. B. für Logging)
queue.on(errorMonitor, (err) => log(err));

// Async Listener: abgelehnte Promises landen sonst im Nirgendwo
class OrderQueue extends EventEmitter {
  constructor() { super({ captureRejections: true }); }
}
queue.on('order', async (order) => { throw new Error('ungültig'); });
queue.emit('order', { id: 99 });
// → landet im 'error'-Handler
```

▸ examples/02-core/02-events/events.js · Schritt 2 + 5

Events als Promise und Async Iterator

```
// Auf genau ein Event warten
const [what] = await once(queue, 'ready');

// Alle Events als Stream konsumieren – bis 'closed' kommt
for await (const [order] of on(queue, 'order', { close: ['closed'] })) {
  console.log('verarbeitet:', order.id);
}

// Web-Pendant: EventTarget
const target = new EventTarget();
target.addEventListener('ping', (e) => console.log(e.detail), { once: true });
```

▸ examples/02-core/02-events/events.js · Schritt 3, 4, 6

Events als Promise und Async Iterator

```
// Auf genau ein Event warten
const [what] = await once(queue, 'ready');

// Alle Events als Stream konsumieren – bis 'closed' kommt
for await (const [order] of on(queue, 'order', { close: ['closed'] })) {
  console.log('verarbeitet:', order.id);
}

// Web-Pendant: EventTarget
const target = new EventTarget();
target.addEventListener('ping', (e) => console.log(e.detail), { once: true });
```

▸ examples/02-core/02-events/events.js · Schritt 3, 4, 6

Events als Promise und Async Iterator

```
// Auf genau ein Event warten
const [what] = await once(queue, 'ready');

// Alle Events als Stream konsumieren – bis 'closed' kommt
for await (const [order] of on(queue, 'order', { close: ['closed'] })) {
  console.log('verarbeitet:', order.id);
}

// Web-Pendant: EventTarget
const target = new EventTarget();
target.addEventListener('ping', (e) => console.log(e.detail), { once: true });
```

▸ examples/02-core/02-events/events.js · Schritt 3, 4, 6

Events als Promise und Async Iterator

```js
// Auf genau ein Event warten
const [what] = await once(queue, 'ready');

// Alle Events als Stream konsumieren – bis 'closed' kommt
for await (const [order] of on(queue, 'order', { close: ['closed'] })) {
  console.log('verarbeitet:', order.id);
}

// Web-Pendant: EventTarget
const target = new EventTarget();
target.addEventListener('ping', (e) => console.log(e.detail), { once: true });
```

`EventEmitter`	Node-API, Server-Code, Streams basieren darauf
`EventTarget`	Web-Standard, isomorpher Code, `AbortSignal`

▸ `examples/02-core/02-events/events.js` · Schritt 3, 4, 6

Streams

Daten verarbeiten, ohne sie komplett im Speicher zu halten

Readable	Datenquelle	`fs.createReadStream`, `req`, `Readable.from()`
Writable	Datensenke	`fs.createWriteStream`, `res`, `process.stdout`
Duplex	beides, unabhängig	TCP-Socket
Transform	beides, verknüpft	`zlib.createGzip()`, Async-Generator

> **Speicher bleibt konstant**
> 14 MB Logdatei filtern und komprimieren – egal ob 14 MB oder 14 GB, der Speicherverbrauch bleibt bei ~64 KiB Puffer pro Stream.

pipeline() + Async-Generatoren

```js
import { pipeline } from 'node:stream/promises';

await pipeline(
  Readable.from(generateLogLines(300_000)),
  createWriteStream('access.log'),
);

await pipeline(
  createReadStream('access.log', { encoding: 'utf8' }),
  splitLines,
  async function* onlyErrors(lines) {
    for await (const line of lines) {
      if (line.includes(' 500 ')) yield line + '\n';
    }
  },
  createGzip(),
  createWriteStream('errors.log.gz'),
);
```

▸ examples/02-core/03-streams/pipeline.js · Schritt 1–2

pipeline() + Async-Generatoren

```
import { pipeline } from 'node:stream/promises';

await pipeline(
  Readable.from(generateLogLines(300_000)),
  createWriteStream('access.log'),
);

await pipeline(
  createReadStream('access.log', { encoding: 'utf8' }),
  splitLines,
  async function* onlyErrors(lines) {
    for await (const line of lines) {
      if (line.includes(' 500 ')) yield line + '\n';
    }
  },
  createGzip(),
  createWriteStream('errors.log.gz'),
);
```

▸ examples/02-core/03-streams/pipeline.js · Schritt 1–2

pipeline() + Async-Generatoren

```
import { pipeline } from 'node:stream/promises';

await pipeline(
  Readable.from(generateLogLines(300_000)),
  createWriteStream('access.log'),
);

await pipeline(
  createReadStream('access.log', { encoding: 'utf8' }),
  splitLines,
  async function* onlyErrors(lines) {
    for await (const line of lines) {
      if (line.includes(' 500 ')) yield line + '\n';
    }
  },
  createGzip(),
  createWriteStream('errors.log.gz'),
);
```

▸ examples/02-core/03-streams/pipeline.js · Schritt 1–2

pipeline() + Async-Generatoren

```js
import { pipeline } from 'node:stream/promises';

await pipeline(
  Readable.from(generateLogLines(300_000)),
  createWriteStream('access.log'),
);

await pipeline(
  createReadStream('access.log', { encoding: 'utf8' }),
  splitLines,
  async function* onlyErrors(lines) {
    for await (const line of lines) {
      if (line.includes(' 500 ')) yield line + '\n';
    }
  },
  createGzip(),
  createWriteStream('errors.log.gz'),
);
```

▸ examples/02-core/03-streams/pipeline.js · Schritt 1–2

Abbrechen & Async Iteration

```
// Alle Streams werden bei Abbruch sauber zerstört
await pipeline(
  createReadStream('access.log'),
  createGzip({ level: 9 }),
  createWriteStream('aborted.gz'),
  { signal: AbortSignal.timeout(5) },
);   // AbortError

// Jeder Readable ist ein Async Iterable
for await (const line of splitLines(createReadStream('access.log', 'utf8'))) {
  const status = line.split(' ')[3];
  stats.set(status, (stats.get(status) ?? 0) + 1);
}
```

▸ examples/02-core/03-streams/pipeline.js · Schritt 3–4

Abbrechen & Async Iteration

```
// Alle Streams werden bei Abbruch sauber zerstört
await pipeline(
  createReadStream('access.log'),
  createGzip({ level: 9 }),
  createWriteStream('aborted.gz'),
  { signal: AbortSignal.timeout(5) },
);   // AbortError

// Jeder Readable ist ein Async Iterable
for await (const line of splitLines(createReadStream('access.log', 'utf8'))) {
  const status = line.split(' ')[3];
  stats.set(status, (stats.get(status) ?? 0) + 1);
}
```

▸ examples/02-core/03-streams/pipeline.js · Schritt 3–4

Backpressure

```
// ❌ ignoriert Backpressure
for (let i = 0; i < 100; i++) {
  writer.write({ i });
}
// max. gepuffert = 100

// ✅ respektiert Backpressure
for (let i = 0; i < 100; i++) {
  if (!writer.write({ i })) {
    await once(writer, 'drain');
  }
}
// max. gepuffert = 4
```

▸ examples/02-core/03-streams/backpressure.js · Schritt 1–2

- `write()` liefert `false`, wenn der Puffer über `highWaterMark` liegt
- Der Producer sollte dann auf `'drain'` warten
- Default `highWaterMark`: 64 KiB 22+
- `pipeline()` und `for await` machen das automatisch

Web Streams & Interop

```
const compressed = source
  .pipeThrough(new TransformStream({ transform: (c, ctl) => ctl.enqueue(c.toUpperCase()) }))
  .pipeThrough(new TextEncoderStream())
  .pipeThrough(new CompressionStream('gzip'));

// Node → Web: z. B. Datei als fetch-Response
const webStream = Readable.toWeb(createReadStream(file));
const response = new Response(webStream);

// Web → Node
const nodeStream = Readable.fromWeb(response.body);
```

> **node:stream/consumers**
>
> `await text(stream)`, `await json(req)`, `await buffer(stream)` – Streams bequem einsammeln.

▸ examples/02-core/03-streams/web-streams.js · Schritt 1–3

Web Streams & Interop

```
const compressed = source
  .pipeThrough(new TransformStream({ transform: (c, ctl) => ctl.enqueue(c.toUpperCase()) }))
  .pipeThrough(new TextEncoderStream())
  .pipeThrough(new CompressionStream('gzip'));

// Node → Web: z. B. Datei als fetch-Response
const webStream = Readable.toWeb(createReadStream(file));
const response = new Response(webStream);

// Web → Node
const nodeStream = Readable.fromWeb(response.body);
```

> **node:stream/consumers**
>
> `await text(stream)`, `await json(req)`, `await buffer(stream)` – Streams bequem einsammeln.

▸ examples/02-core/03-streams/web-streams.js · Schritt 1–3

Web Streams & Interop

```
const compressed = source
  .pipeThrough(new TransformStream({ transform: (c, ctl) => ctl.enqueue(c.toUpperCase()) }))
  .pipeThrough(new TextEncoderStream())
  .pipeThrough(new CompressionStream('gzip'));

// Node → Web: z. B. Datei als fetch-Response
const webStream = Readable.toWeb(createReadStream(file));
const response = new Response(webStream);

// Web → Node
const nodeStream = Readable.fromWeb(response.body);
```

node:stream/consumers

`await text(stream)`, `await json(req)`, `await buffer(stream)` – Streams bequem einsammeln.

▸ examples/02-core/03-streams/web-streams.js · Schritt 1–3

Worker Threads

Für CPU-Last – nicht für I/O

```
function runInWorker(n) {
  return new Promise((resolve, reject) => {
    const worker = new Worker(new URL('./fib-worker.js', import.meta.url),
      { workerData: n });
    worker.once('message', resolve);
    worker.once('error', reject);
  });
}

const result = await runInWorker(40);
```

```
.....                       ← Ticker läuft …
Main Thread: 637ms          ← … und steht während fib(40)
...........                 ← Ticker läuft weiter
Worker: 649ms
```

▸ examples/02-core/04-workers/main.js · Schritt 1–2

Worker Threads

Für CPU-Last – nicht für I/O

```
function runInWorker(n) {
  return new Promise((resolve, reject) => {
    const worker = new Worker(new URL('./fib-worker.js', import.meta.url),
      { workerData: n });
    worker.once('message', resolve);
    worker.once('error', reject);
  });
}

const result = await runInWorker(40);
```

```
.....                       ← Ticker läuft …
Main Thread: 637ms          ← … und steht während fib(40)
...........                 ← Ticker läuft weiter
Worker: 649ms
```

▸ examples/02-core/04-workers/main.js · Schritt 1–2

Worker sind teuer – deshalb ein Pool

```
run(data) {
  const { promise, resolve, reject } = Promise.withResolvers();
  this.#queue.push({ data, resolve, reject });
  if (this.#idle.length > 0) this.#next(this.#idle.pop());
  return promise;
}

#next(worker) {
  const task = this.#queue.shift();
  if (!task) {
    this.#idle.push(worker);
    return;
  }
  this.#running.set(worker, task);
  worker.postMessage(task.data);
}
```

```
sequenziell im Main Thread: 423ms
parallel im Pool (4 Worker): 127ms
```

▸ examples/02-core/04-workers/pool.js · Schritt 1–3 · produktiv: piscina

Worker sind teuer – deshalb ein Pool

```
run(data) {
  const { promise, resolve, reject } = Promise.withResolvers();
  this.#queue.push({ data, resolve, reject });
  if (this.#idle.length > 0) this.#next(this.#idle.pop());
  return promise;
}

#next(worker) {
  const task = this.#queue.shift();
  if (!task) {
    this.#idle.push(worker);
    return;
  }
  this.#running.set(worker, task);
  worker.postMessage(task.data);
}
```

```
sequenziell im Main Thread: 423ms
parallel im Pool (4 Worker): 127ms
```

▸ examples/02-core/04-workers/pool.js · Schritt 1–3 · produktiv: piscina

Gemeinsamer Speicher

```
// Main Thread
const shared = new SharedArrayBuffer(8);
new Worker(file, { workerData: shared });

// Worker
const counters = new Int32Array(workerData);
counters[0]++;                  // Race Condition
Atomics.add(counters, 1, 1);    // atomar
```

```
erwartet:       4000000
ohne Atomics:   3909605
mit Atomics:    4000000
```

`postMessage()`	kopiert Daten (Structured Clone)
`postMessage(data, [buffer])`	überträgt Besitz eines ArrayBuffers ohne Kopie
`SharedArrayBuffer`	teilt Speicher – braucht `Atomics`
`MessageChannel`, `BroadcastChannel`	direkte Kanäle zwischen Threads

▸ examples/02-core/04-workers/shared-memory.js

Async-Kontrollfluss

Sequenziell vs. parallel

```
for (const id of [1, 2, 3]) await fetchUser(id);          // 328ms

await Promise.all([1, 2, 3].map((id) => fetchUser(id)));   // 108ms
```

Kombinator	erfüllt, wenn …	abgelehnt, wenn …
`Promise.all`	alle erfüllt	einer abgelehnt (sofort)
`Promise.allSettled`	alle fertig	nie
`Promise.any`	einer erfüllt	alle abgelehnt (`AggregateError`)
`Promise.race`	der erste fertige erfüllt	der erste fertige abgelehnt

▸ `examples/02-core/05-async-flow/control-flow.js · Schritt 1–2`

Abbruch durchreichen

```
async function loadDashboard({ signal }) {
  return Promise.all([
    fetchUser(1, { signal }),
    fetchOrders(1, { signal }),
  ]);
}

const controller = new AbortController();
cancelButton.on('click', () => controller.abort(new Error('Benutzer hat abgebrochen')));

// Mehrere Abbruchgründe: manuell ODER Timeout
const signal = AbortSignal.any([controller.signal, AbortSignal.timeout(1000)]);

try {
  await loadDashboard({ signal });
} catch (err) {
  console.log(err.name, signal.reason.message);
}
```

> **Regel**
>
> Jede async Funktion, die I/O macht, nimmt ein `signal` an und reicht es weiter – `fetch`, `timers/promises`, `pipeline`, `events.once` verstehen es.

▸ examples/02-core/05-async-flow/control-flow.js · Schritt 3

Abbruch durchreichen

```
async function loadDashboard({ signal }) {
  return Promise.all([
    fetchUser(1, { signal }),
    fetchOrders(1, { signal }),
  ]);
}

const controller = new AbortController();
cancelButton.on('click', () => controller.abort(new Error('Benutzer hat abgebrochen')));

// Mehrere Abbruchgründe: manuell ODER Timeout
const signal = AbortSignal.any([controller.signal, AbortSignal.timeout(1000)]);

try {
  await loadDashboard({ signal });
} catch (err) {
  console.log(err.name, signal.reason.message);
}
```

> **Regel**
>
> Jede async Funktion, die I/O macht, nimmt ein `signal` an und reicht es weiter – `fetch`, `timers/promises`, `pipeline`, `events.once` verstehen es.

▸ examples/02-core/05-async-flow/control-flow.js · Schritt 3

Abbruch durchreichen

```
async function loadDashboard({ signal }) {
  return Promise.all([
    fetchUser(1, { signal }),
    fetchOrders(1, { signal }),
  ]);
}

const controller = new AbortController();
cancelButton.on('click', () => controller.abort(new Error('Benutzer hat abgebrochen')));

// Mehrere Abbruchgründe: manuell ODER Timeout
const signal = AbortSignal.any([controller.signal, AbortSignal.timeout(1000)]);

try {
  await loadDashboard({ signal });
} catch (err) {
  console.log(err.name, signal.reason.message);
}
```

> **Regel**
>
> Jede async Funktion, die I/O macht, nimmt ein `signal` an und reicht es weiter – `fetch`, `timers/promises`, `pipeline`, `events.once` verstehen es.

▸ examples/02-core/05-async-flow/control-flow.js · Schritt 3

Abbruch durchreichen

```
async function loadDashboard({ signal }) {
  return Promise.all([
    fetchUser(1, { signal }),
    fetchOrders(1, { signal }),
  ]);
}

const controller = new AbortController();
cancelButton.on('click', () => controller.abort(new Error('Benutzer hat abgebrochen')));

// Mehrere Abbruchgründe: manuell ODER Timeout
const signal = AbortSignal.any([controller.signal, AbortSignal.timeout(1000)]);

try {
  await loadDashboard({ signal });
} catch (err) {
  console.log(err.name, signal.reason.message);
}
```

Regel

Jede async Funktion, die I/O macht, nimmt ein `signal` an und reicht es weiter – `fetch`, `timers/promises`, `pipeline`, `events.once` verstehen es.

▸ examples/02-core/05-async-flow/control-flow.js · Schritt 3

Kleine Helfer mit großer Wirkung

```js
// Promise.withResolvers – Promise und Steuerung getrennt
const { promise: connected, resolve: onConnect } = Promise.withResolvers();
socket.on('connect', () => onConnect('Verbindung steht'));
await connected;

// Async Generator + Array.fromAsync – z. B. Paginierung
async function* paginate(pages) {
  for (let page = 1; page <= pages; page++) {
    const items = await fetchPage(page);
    yield* items;
  }
}

const all = await Array.fromAsync(paginate(3));
```

▸ examples/02-core/05-async-flow/control-flow.js · Schritt 4–5

Kleine Helfer mit großer Wirkung

```
// Promise.withResolvers – Promise und Steuerung getrennt
const { promise: connected, resolve: onConnect } = Promise.withResolvers();
socket.on('connect', () => onConnect('Verbindung steht'));
await connected;

// Async Generator + Array.fromAsync – z. B. Paginierung
async function* paginate(pages) {
  for (let page = 1; page <= pages; page++) {
    const items = await fetchPage(page);
    yield* items;
  }
}

const all = await Array.fromAsync(paginate(3));
```

▸ examples/02-core/05-async-flow/control-flow.js · Schritt 4–5

Teil 3

Architektur für robuste Services

Module · Schichten · Fehler · Concurrency · Observability

Der Task-Service

Alles aus diesem Workshop in einer App

```
04-service/
├── package.json    imports, exports, scripts
├── .env
├── src/
│   ├── main.js     Prozess-Lifecycle
│   ├── app.js      Composition Root
│   ├── config.js
│   ├── lib/
│   │   ├── context.js   AsyncLocalStorage
│   │   ├── errors.js
│   │   ├── http.js      Router, Error-Mapping
│   │   └── logger.js
│   └── tasks/
│       ├── task.routes.js
│       ├── task.service.js
│       └── task.repository.js
└── test/
```

▸ examples/03-architecture/04-service

Routes – HTTP ⇄ Service
kennt Request-Format, Status-Codes

Service – Fachlogik
kennt weder HTTP noch SQL

Repository – Datenzugriff
kapselt SQL komplett

Modulstruktur mit package.json

```
{
  "name": "task-service",
  "type": "module",
  "imports": {
    "#lib/*": "./src/lib/*.js",
    "#tasks/*": "./src/tasks/*.js"
  },
  "exports": {
    ".": "./src/app.js"
  }
}
```

```
// statt '../../lib/errors.js'
import { NotFoundError } from '#lib/errors';
import { TaskService } from '#tasks/task.service';

// Self-Reference über "exports"
import { createApp } from 'task-service';
```

- `imports` (`#...`): private Aliase, nur innerhalb des Pakets
- `exports`: öffentliche API – alles andere ist von außen nicht importierbar
- Kein Bundler-/TS-Path-Mapping nötig, funktioniert überall

Handler als reine Funktionen

```
router.get('/tasks/:id', async ({ params }) => ({
  body: await taskService.get(parseId(params.id)),
}));

router.post('/tasks', async ({ body }) => ({
  status: 201,
  body: await taskService.create(body),
}));

// Routing mit dem Web-Standard URLPattern (24+)
const pattern = new URLPattern({ pathname: '/tasks/:id' });
pattern.exec({ pathname: '/tasks/42' }).pathname.groups;   // { id: '42' }
```

> **Handler kennen weder `req` noch `res`**
>
> Eingabe rein, Daten raus – einfach testbar, Framework austauschbar.

▸ src/tasks/task.routes.js · src/lib/http.js · Schritt 1–2

Handler als reine Funktionen

```
router.get('/tasks/:id', async ({ params }) => ({
  body: await taskService.get(parseId(params.id)),
}));

router.post('/tasks', async ({ body }) => ({
  status: 201,
  body: await taskService.create(body),
}));

// Routing mit dem Web-Standard URLPattern (24+)
const pattern = new URLPattern({ pathname: '/tasks/:id' });
pattern.exec({ pathname: '/tasks/42' }).pathname.groups;   // { id: '42' }
```

> **Handler kennen weder `req` noch `res`**
> Eingabe rein, Daten raus – einfach testbar, Framework austauschbar.

▸ src/tasks/task.routes.js · src/lib/http.js · Schritt 1–2

Handler als reine Funktionen

```
router.get('/tasks/:id', async ({ params }) => ({
  body: await taskService.get(parseId(params.id)),
}));

router.post('/tasks', async ({ body }) => ({
  status: 201,
  body: await taskService.create(body),
}));

// Routing mit dem Web-Standard URLPattern (24+)
const pattern = new URLPattern({ pathname: '/tasks/:id' });
pattern.exec({ pathname: '/tasks/42' }).pathname.groups;   // { id: '42' }
```

> **Handler kennen weder `req` noch `res`**
> Eingabe rein, Daten raus – einfach testbar, Framework austauschbar.

▸ src/tasks/task.routes.js · src/lib/http.js · Schritt 1–2

Service & Repository

```
export class TaskService {
  #repository;
  #logger;

  constructor({ taskRepository, logger }) {
    this.#repository = taskRepository;
    this.#logger = logger;
  }

  async get(id) {
    const task = await this.#repository.findById(id);
    if (!task) {
      throw new NotFoundError(`Task ${id} nicht gefunden`);
    }
    return task;
  }
}
```

```
import { DatabaseSync } from 'node:sqlite';

export class SqliteTaskRepository {
  #statements;

  constructor(db) {
    db.exec(schema);
    this.#statements = {
      findById: db.prepare(
        'SELECT * FROM tasks WHERE id = ?'),
    };
  }

  // async, obwohl sqlite sync ist:
  // Interface bleibt stabil
  async findById(id) {
    const row = this.#statements.findById.get(id);
    return row && toTask(row);
  }
}
```

▸ src/tasks/task.service.js · task.repository.js

Composition Root

```
export function createApp({ config, logger }) {
  const db = new DatabaseSync(config.dbFile);
  const taskRepository = new SqliteTaskRepository(db);
  const taskService = new TaskService({ taskRepository, logger });

  const router = createRouter({ logger });
  router.get('/health', () => ({ body: { status: 'ok' } }));
  registerTaskRoutes(router, { taskService });

  return { server: createServer(router.handler), close };
}
```

- Nur hier werden konkrete Implementierungen verdrahtet – Dependency Injection ohne Container
- Tests tauschen Teile aus: In-Memory-Repository, stiller Logger, Port 0

▸ src/app.js · Schritt 1

Composition Root

```
export function createApp({ config, logger }) {
  const db = new DatabaseSync(config.dbFile);
  const taskRepository = new SqliteTaskRepository(db);
  const taskService = new TaskService({ taskRepository, logger });

  const router = createRouter({ logger });
  router.get('/health', () => ({ body: { status: 'ok' } }));
  registerTaskRoutes(router, { taskService });

  return { server: createServer(router.handler), close };
}
```

- Nur hier werden konkrete Implementierungen verdrahtet – Dependency Injection ohne Container
- Tests tauschen Teile aus: In-Memory-Repository, stiller Logger, Port 0

▸ src/app.js · Schritt 1

Composition Root

```
export function createApp({ config, logger }) {
  const db = new DatabaseSync(config.dbFile);
  const taskRepository = new SqliteTaskRepository(db);
  const taskService = new TaskService({ taskRepository, logger });

  const router = createRouter({ logger });
  router.get('/health', () => ({ body: { status: 'ok' } }));
  registerTaskRoutes(router, { taskService });

  return { server: createServer(router.handler), close };
}
```

- Nur hier werden konkrete Implementierungen verdrahtet – Dependency Injection ohne Container
- Tests tauschen Teile aus: In-Memory-Repository, stiller Logger, Port 0

▸ src/app.js · Schritt 1

Error-Handling-Strategie

	Operational Errors	Programmer Errors
Beispiele	404, Validierung, Timeout, DB nicht erreichbar	`TypeError`, `undefined is not a function`
Ursache	Umgebung, Eingaben	Bug im Code
Umgang	erwarten, behandeln, sinnvoll antworten	loggen, Prozess sterben lassen, neu starten
Nach außen	Code + verständliche Nachricht	generisches 500, keine Interna

▸ `examples/03-architecture/01-errors/errors.js`

Fehlerklassen & error.cause

```
class AppError extends Error {
  constructor(message, { code = 'INTERNAL', status = 500, cause, details } = {}) {
    super(message, cause === undefined ? undefined : { cause });
    this.name = new.target.name;
    this.code = code;
    this.status = status;
    if (details) this.details = details;
  }
}

class NotFoundError extends AppError {
  constructor(resource, id) {
    super(`${resource} ${id} nicht gefunden`, { code: 'NOT_FOUND', status: 404 });
  }
}

async function loadConfig(path) {
  try {
    return JSON.parse(await readFile(path, 'utf8'));
  } catch (err) {
    throw new AppError(`Konfiguration "${path}" nicht ladbar`, { code: 'CONFIG', cause: err });
  }
}
```

▸ examples/03-architecture/01-errors/errors.js · Schritt 1–2

Fehlerklassen & error.cause

```
class AppError extends Error {
  constructor(message, { code = 'INTERNAL', status = 500, cause, details } = {}) {
    super(message, cause === undefined ? undefined : { cause });
    this.name = new.target.name;
    this.code = code;
    this.status = status;
    if (details) this.details = details;
  }
}

class NotFoundError extends AppError {
  constructor(resource, id) {
    super(`${resource} ${id} nicht gefunden`, { code: 'NOT_FOUND', status: 404 });
  }
}

async function loadConfig(path) {
  try {
    return JSON.parse(await readFile(path, 'utf8'));
  } catch (err) {
    throw new AppError(`Konfiguration "${path}" nicht ladbar`, { code: 'CONFIG', cause: err });
  }
}
```

▸ examples/03-architecture/01-errors/errors.js · Schritt 1–2

Fehlerklassen & error.cause

```
class AppError extends Error {
  constructor(message, { code = 'INTERNAL', status = 500, cause, details } = {}) {
    super(message, cause === undefined ? undefined : { cause });
    this.name = new.target.name;
    this.code = code;
    this.status = status;
    if (details) this.details = details;
  }
}

class NotFoundError extends AppError {
  constructor(resource, id) {
    super(`${resource} ${id} nicht gefunden`, { code: 'NOT_FOUND', status: 404 });
  }
}

async function loadConfig(path) {
  try {
    return JSON.parse(await readFile(path, 'utf8'));
  } catch (err) {
    throw new AppError(`Konfiguration "${path}" nicht ladbar`, { code: 'CONFIG', cause: err });
  }
}
```

▸ examples/03-architecture/01-errors/errors.js · Schritt 1–2

Fehler zentral übersetzen

```
function toErrorResponse(err) {
  if (err instanceof AppError) {
    return {
      status: err.status,
      body: { error: err.code, message: err.message, details: err.details },
    };
  }
  logger.error('unhandled error', { err });
  return { status: 500, body: { error: 'INTERNAL', message: 'Internal Server Error' } };
}

const response = await dispatch(req).catch(toErrorResponse);
```

```
GET /tasks/99   → 404 {"error":"NOT_FOUND","message":"Task 99 nicht gefunden"}
POST /tasks {}  → 400 {"error":"VALIDATION","message":"title ist erforderlich",…}
```

▸ src/lib/http.js · Schritt 3

Fehler zentral übersetzen

```
function toErrorResponse(err) {
  if (err instanceof AppError) {
    return {
      status: err.status,
      body: { error: err.code, message: err.message, details: err.details },
    };
  }
  logger.error('unhandled error', { err });
  return { status: 500, body: { error: 'INTERNAL', message: 'Internal Server Error' } };
}

const response = await dispatch(req).catch(toErrorResponse);
```

```
GET /tasks/99   → 404 {"error":"NOT_FOUND","message":"Task 99 nicht gefunden"}
POST /tasks {}  → 400 {"error":"VALIDATION","message":"title ist erforderlich",…}
```

▸ src/lib/http.js · Schritt 3

Ressourcen garantiert freigeben 24+

```
async function openConnection(name) {
  return {
    query: async (sql) => { /* … */ },
    async [Symbol.asyncDispose]() { console.log(`${name}: geschlossen`); },
  };
}

async function report() {
  await using db = await openConnection('db');
  return db.query('SELECT * FROM orders');  // wirft? db wird trotzdem geschlossen
}
```

Explicit Resource Management: `using` / `await using` ersetzen `try … finally` für Verbindungen, Datei-Handles, Locks, Timer.

▸ examples/03-architecture/01-errors/errors.js · Schritt 4

Ressourcen garantiert freigeben 24+

```
async function openConnection(name) {
  return {
    query: async (sql) => { /* … */ },
    async [Symbol.asyncDispose]() { console.log(`${name}: geschlossen`); },
  };
}

async function report() {
  await using db = await openConnection('db');
  return db.query('SELECT * FROM orders');  // wirft? db wird trotzdem geschlossen
}
```

Explicit Resource Management: `using` / `await using` ersetzen `try … finally` für Verbindungen, Datei-Handles, Locks, Timer.

▸ examples/03-architecture/01-errors/errors.js · Schritt 4

Die letzte Verteidigungslinie

```
// Unbehandelte Rejections beenden den Prozess (Default seit Node 15)
process.on('unhandledRejection', (reason) => {
  /* nur, wenn ihr wirklich anders reagieren wollt */
});

// Besser: nur loggen - Node beendet den Prozess danach trotzdem
process.on('uncaughtExceptionMonitor', (err, origin) => {
  logger.error('fatal error', { origin, err });
});
```

Let it crash

Nach einem Programmierfehler ist der Zustand unbekannt. Sauber loggen, beenden und vom Supervisor (systemd, Docker, Kubernetes) neu starten lassen.

▸ examples/03-architecture/01-errors/errors.js · Schritt 5 · src/main.js · Schritt 2

Die letzte Verteidigungslinie

```
// Unbehandelte Rejections beenden den Prozess (Default seit Node 15)
process.on('unhandledRejection', (reason) => {
  /* nur, wenn ihr wirklich anders reagieren wollt */
});

// Besser: nur loggen – Node beendet den Prozess danach trotzdem
process.on('uncaughtExceptionMonitor', (err, origin) => {
  logger.error('fatal error', { origin, err });
});
```

Let it crash

Nach einem Programmierfehler ist der Zustand unbekannt. Sauber loggen, beenden und vom Supervisor (systemd, Docker, Kubernetes) neu starten lassen.

▸ examples/03-architecture/01-errors/errors.js · Schritt 5 · src/main.js · Schritt 2

Concurrency-Patterns

Downstream-Systeme schützen

```
function createLimiter(max) {
  let running = 0;
  const queue = [];
  const next = () => {
    if (running >= max || queue.length === 0) return;
    running++;
    const { task, resolve, reject } = queue.shift();
    task().then(resolve, reject).finally(() => { running--; next(); });
  };
  return (task) => {
    const { promise, resolve, reject } = Promise.withResolvers();
    queue.push({ task, resolve, reject });
    next();
    return promise;
  };
}

const limit = createLimiter(5);
await Promise.all(ids.map((id) => limit(() => callApi(id))));   // max. 5 gleichzeitig
```

▸ examples/03-architecture/02-concurrency/patterns.js · Schritt 1

Concurrency-Patterns

Downstream-Systeme schützen

```
function createLimiter(max) {
  let running = 0;
  const queue = [];
  const next = () => {
    if (running >= max || queue.length === 0) return;
    running++;
    const { task, resolve, reject } = queue.shift();
    task().then(resolve, reject).finally(() => { running--; next(); });
  };
  return (task) => {
    const { promise, resolve, reject } = Promise.withResolvers();
    queue.push({ task, resolve, reject });
    next();
    return promise;
  };
}

const limit = createLimiter(5);
await Promise.all(ids.map((id) => limit(() => callApi(id))));   // max. 5 gleichzeitig
```

▸ examples/03-architecture/02-concurrency/patterns.js · Schritt 1

Retry, Timeout, Coalescing

```js
async function retry(task, { retries = 4, baseDelay = 50, signal } = {}) {
  for (let attempt = 0; ; attempt++) {
    try {
      return await task({ attempt, signal });
    } catch (err) {
      if (attempt >= retries || !err.retryable) throw err;
      const delay = baseDelay * 2 ** attempt
        * (0.5 + Math.random() / 2);  // Jitter
      await sleep(delay, undefined, { signal });
    }
  }
}

await retry(callApi, { signal: AbortSignal.timeout(200) });
```

```js
const inflight = new Map();

function coalesce(key, task) {
  if (!inflight.has(key)) {
    inflight.set(key,
      task().finally(() => inflight.delete(key)));
  }
  return inflight.get(key);
}

// 50 parallele Anfragen → 1 echter Aufruf
```

- **Backoff + Jitter** verhindern, dass alle Clients gleichzeitig erneut anfragen
- Das **Timeout** gilt für die gesamte Operation inklusive aller Retries

▸ examples/03-architecture/02-concurrency/patterns.js · Schritt 2–4

Graceful Shutdown

```
async function close() {
  const closed = new Promise((resolve, reject) =>
    server.close((err) => (err ? reject(err) : resolve())));  // keine neuen Verbindungen
  server.closeIdleConnections();
  await closed;                                               // laufende Requests fertig
  db.close();                                                 // dann Ressourcen
}

async function shutdown(signal) {
  logger.info('shutting down', { signal });
  setTimeout(() => process.exit(1), config.shutdownTimeout).unref();   // Notbremse
  await app.close();
  logger.info('shutdown complete');
}

process.once('SIGINT', shutdown);    // Ctrl+C
process.once('SIGTERM', shutdown);   // Docker, Kubernetes
```

▸ src/app.js · Schritt 2 · src/main.js · Schritt 1

Graceful Shutdown

```
async function close() {
  const closed = new Promise((resolve, reject) =>
    server.close((err) => (err ? reject(err) : resolve())));  // keine neuen Verbindungen
  server.closeIdleConnections();
  await closed;                                               // laufende Requests fertig
  db.close();                                                 // dann Ressourcen
}

async function shutdown(signal) {
  logger.info('shutting down', { signal });
  setTimeout(() => process.exit(1), config.shutdownTimeout).unref();   // Notbremse
  await app.close();
  logger.info('shutdown complete');
}

process.once('SIGINT', shutdown);    // Ctrl+C
process.once('SIGTERM', shutdown);   // Docker, Kubernetes
```

▸ src/app.js · Schritt 2 · src/main.js · Schritt 1

Graceful Shutdown

```
async function close() {
  const closed = new Promise((resolve, reject) =>
    server.close((err) => (err ? reject(err) : resolve())));  // keine neuen Verbindungen
  server.closeIdleConnections();
  await closed;                                               // laufende Requests fertig
  db.close();                                                 // dann Ressourcen
}

async function shutdown(signal) {
  logger.info('shutting down', { signal });
  setTimeout(() => process.exit(1), config.shutdownTimeout).unref();   // Notbremse
  await app.close();
  logger.info('shutdown complete');
}

process.once('SIGINT', shutdown);    // Ctrl+C
process.once('SIGTERM', shutdown);   // Docker, Kubernetes
```

▸ src/app.js · Schritt 2 · src/main.js · Schritt 1

Observability mit Bordmitteln

Säule	Frage	Node.js-Baustein
Logs	Was ist passiert?	strukturiertes JSON + `AsyncLocalStorage`
Metriken	Wie oft, wie schnell?	`perf_hooks`: `createHistogram`, `monitorEventLoopDelay`
Traces	Wo geht die Zeit verloren?	`diagnostics_channel`, `TracingChannel`

> **OpenTelemetry**
> Die OTel-Instrumentierungen nutzen genau diese Bausteine. Wer sie versteht, versteht, was der APM-Agent tut.

▸ `examples/03-architecture/03-observability/server.js`

AsyncLocalStorage

Kontext ohne Parameter-Durchreichen

```
const requestContext = new AsyncLocalStorage();

function log(level, msg, data = {}) {
  const context = requestContext.getStore() ?? {};
  console.log(JSON.stringify({ level, msg, requestId: context.requestId, ...data }));
}

const server = createServer((req, res) => {
  const requestId = req.headers['x-request-id'] ?? randomUUID();
  requestContext.run({ requestId }, () => handle(req, res));
});
```

```
{"level":"info","msg":"handling","requestId":"a1"}
{"level":"info","msg":"handling","requestId":"b2"}
{"level":"debug","msg":"db query","requestId":"b2","ms":40}
{"level":"debug","msg":"db query","requestId":"a1","ms":41}
{"level":"info","msg":"loaded","requestId":"b2","rows":1}
```

▸ examples/03-architecture/03-observability/server.js · Schritt 1

AsyncLocalStorage

Kontext ohne Parameter-Durchreichen

```
const requestContext = new AsyncLocalStorage();

function log(level, msg, data = {}) {
  const context = requestContext.getStore() ?? {};
  console.log(JSON.stringify({ level, msg, requestId: context.requestId, ...data }));
}

const server = createServer((req, res) => {
  const requestId = req.headers['x-request-id'] ?? randomUUID();
  requestContext.run({ requestId }, () => handle(req, res));
});
```

```
{"level":"info","msg":"handling","requestId":"a1"}
{"level":"info","msg":"handling","requestId":"b2"}
{"level":"debug","msg":"db query","requestId":"b2","ms":40}
{"level":"debug","msg":"db query","requestId":"a1","ms":41}
{"level":"info","msg":"loaded","requestId":"b2","rows":1}
```

▸ examples/03-architecture/03-observability/server.js · Schritt 1

AsyncLocalStorage

Kontext ohne Parameter-Durchreichen

```
const requestContext = new AsyncLocalStorage();

function log(level, msg, data = {}) {
  const context = requestContext.getStore() ?? {};
  console.log(JSON.stringify({ level, msg, requestId: context.requestId, ...data }));
}

const server = createServer((req, res) => {
  const requestId = req.headers['x-request-id'] ?? randomUUID();
  requestContext.run({ requestId }, () => handle(req, res));
});
```

```
{"level":"info","msg":"handling","requestId":"a1"}
{"level":"info","msg":"handling","requestId":"b2"}
{"level":"debug","msg":"db query","requestId":"b2","ms":40}
{"level":"debug","msg":"db query","requestId":"a1","ms":41}
{"level":"info","msg":"loaded","requestId":"b2","rows":1}
```

▸ examples/03-architecture/03-observability/server.js · Schritt 1

diagnostics_channel

Instrumentierung entkoppeln

```
const dbChannel = diagnosticsChannel.tracingChannel('app:db.query');

async function query(sql) {
  return dbChannel.tracePromise(() => db.all(sql), { sql });
}

// Irgendwo anders - z. B. im APM-Agent
dbChannel.subscribe({
  start(ctx) { ctx.startedAt = performance.now(); },
  asyncEnd(ctx) {
    log('debug', 'db query', { sql: ctx.sql, ms: performance.now() - ctx.startedAt });
  },
  error(ctx) { log('error', 'db query failed', { error: ctx.error.message }); },
});
```

Ohne Subscriber kostet ein Channel praktisch nichts – ideal für Libraries.

▸ examples/03-architecture/03-observability/server.js · Schritt 2

diagnostics_channel

Instrumentierung entkoppeln

```
const dbChannel = diagnosticsChannel.tracingChannel('app:db.query');

async function query(sql) {
  return dbChannel.tracePromise(() => db.all(sql), { sql });
}

// Irgendwo anders – z. B. im APM-Agent
dbChannel.subscribe({
  start(ctx) { ctx.startedAt = performance.now(); },
  asyncEnd(ctx) {
    log('debug', 'db query', { sql: ctx.sql, ms: performance.now() - ctx.startedAt });
  },
  error(ctx) { log('error', 'db query failed', { error: ctx.error.message }); },
});
```

Ohne Subscriber kostet ein Channel praktisch nichts – ideal für Libraries.

▸ examples/03-architecture/03-observability/server.js · Schritt 2

Built-in Channels & Metriken

```
diagnosticsChannel.subscribe('http.server.request.start', ({ request }) => {
  request.startedAt = performance.now();
});

const requestDurations = createHistogram();
diagnosticsChannel.subscribe('http.server.response.finish', ({ request }) => {
  requestDurations.record(Math.round(performance.now() - request.startedAt));
});

console.log({
  requests: requestDurations.count,
  p50: requestDurations.percentile(50),
  p99: requestDurations.percentile(99),
});
```

Weitere Channels u. a.: `http.client.*`, `undici:request:*` (fetch), `net.client.socket`, `worker_threads`, `module.import`

▸ `examples/03-architecture/03-observability/server.js · Schritt 3–4`

Built-in Channels & Metriken

```
diagnosticsChannel.subscribe('http.server.request.start', ({ request }) => {
  request.startedAt = performance.now();
});

const requestDurations = createHistogram();
diagnosticsChannel.subscribe('http.server.response.finish', ({ request }) => {
  requestDurations.record(Math.round(performance.now() - request.startedAt));
});

console.log({
  requests: requestDurations.count,
  p50: requestDurations.percentile(50),
  p99: requestDurations.percentile(99),
});
```

Weitere Channels u. a.: `http.client.*`, `undici:request:*` (fetch), `net.client.socket`, `worker_threads`, `module.import`

▸ `examples/03-architecture/03-observability/server.js · Schritt 3–4`

Built-in Channels & Metriken

```
diagnosticsChannel.subscribe('http.server.request.start', ({ request }) => {
  request.startedAt = performance.now();
});

const requestDurations = createHistogram();
diagnosticsChannel.subscribe('http.server.response.finish', ({ request }) => {
  requestDurations.record(Math.round(performance.now() - request.startedAt));
});

console.log({
  requests: requestDurations.count,
  p50: requestDurations.percentile(50),
  p99: requestDurations.percentile(99),
});
```

Weitere Channels u. a.: `http.client.*`, `undici:request:*` (fetch), `net.client.socket`, `worker_threads`, `module.import`

▸ `examples/03-architecture/03-observability/server.js · Schritt 3–4`

Architektur testen

```
// Unit: Service mit In-Memory-Repository
class InMemoryTaskRepository {
  #tasks = new Map();
  async findById(id) { return this.#tasks.get(id); }
  /* … */
}

const service = new TaskService({
  taskRepository: new InMemoryTaskRepository(),
  logger: silentLogger,
});
await assert.rejects(service.get(42), NotFoundError);
```

```
// Integration: echte App auf freiem Port
before(async () => {
  app = createApp({
    config: { dbFile: ':memory:' },
    logger: silentLogger,
  });
  app.server.listen(0);
  await once(app.server, 'listening');
});

after(() => app.close());
```

```
$ node --run test
i tests 11  i pass 11  i fail 0  i duration_ms 129
```

▸ examples/03-architecture/04-service/test

Die Prinzipien

Die Prinzipien

1. **Plattform zuerst –** was Node kann, braucht keine Dependency

Die Prinzipien

1. **Plattform zuerst –** was Node kann, braucht keine Dependency
2. **Event Loop frei halten –** messen, aufteilen, Worker für CPU-Last

Die Prinzipien

1. **Plattform zuerst –** was Node kann, braucht keine Dependency
2. **Event Loop frei halten –** messen, aufteilen, Worker für CPU-Last
3. **Streams für Daten –** `pipeline()` und Async-Generatoren

Die Prinzipien

1. **Plattform zuerst –** was Node kann, braucht keine Dependency
2. **Event Loop frei halten –** messen, aufteilen, Worker für CPU-Last
3. **Streams für Daten –** `pipeline()` und Async-Generatoren
4. **Abbruch durchreichen –** `AbortSignal` in jede async API

Die Prinzipien

1. **Plattform zuerst –** was Node kann, braucht keine Dependency
2. **Event Loop frei halten –** messen, aufteilen, Worker für CPU-Last
3. **Streams für Daten –** `pipeline()` und Async-Generatoren
4. **Abbruch durchreichen –** `AbortSignal` in jede async API
5. **Fehler sind Teil des Designs –** klassifizieren, anreichern, zentral übersetzen

Die Prinzipien

1. **Plattform zuerst** – was Node kann, braucht keine Dependency
2. **Event Loop frei halten** – messen, aufteilen, Worker für CPU-Last
3. **Streams für Daten** – `pipeline()` und Async-Generatoren
4. **Abbruch durchreichen** – `AbortSignal` in jede async API
5. **Fehler sind Teil des Designs** – klassifizieren, anreichern, zentral übersetzen
6. **Klare Schichten** – Handler, Service, Repository, eine Composition Root

Die Prinzipien

1. **Plattform zuerst** – was Node kann, braucht keine Dependency
2. **Event Loop frei halten** – messen, aufteilen, Worker für CPU-Last
3. **Streams für Daten** – `pipeline()` und Async-Generatoren
4. **Abbruch durchreichen** – `AbortSignal` in jede async API
5. **Fehler sind Teil des Designs** – klassifizieren, anreichern, zentral übersetzen
6. **Klare Schichten** – Handler, Service, Repository, eine Composition Root
7. **Beobachtbar bauen** – Kontext, Channels, Metriken von Anfang an

Weiterlesen

- nodejs.org/api – die API-Doku, inkl. Stabilitätsindex pro Feature
- nodejs.org/learn – Guides zu Event Loop, Profiling, Test Runner, TypeScript
- Node.js Changelog – was kommt mit welcher Version
- Permission Model · diagnostics_channel · TypeScript
- Bewährte Pakete, die auf diesen Bausteinen aufsetzen: **piscina** (Worker Pools), **pino** (Logging), **OpenTelemetry JS** (Tracing)



Danke!

Fragen?